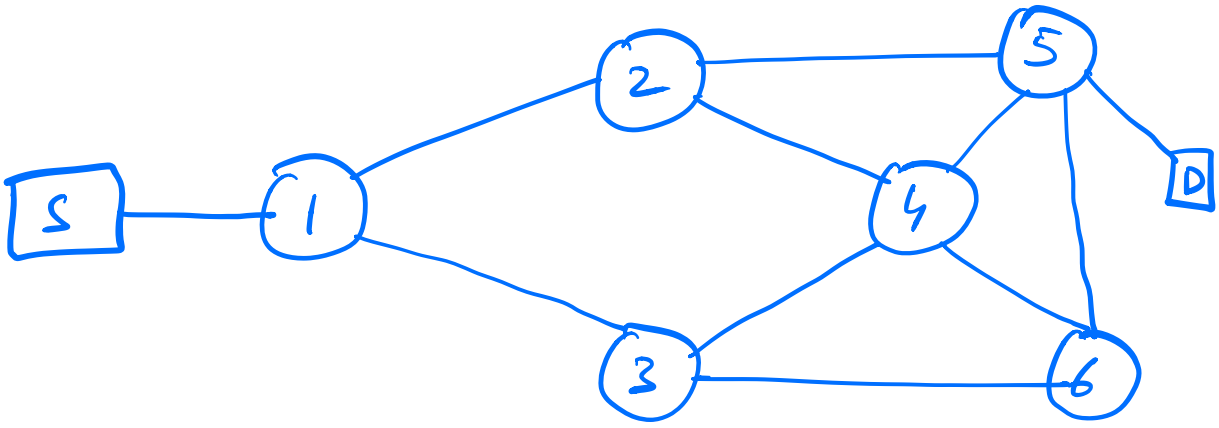
Function Approximation Methods
(Chapter 9)
+
Deep Q-Network
- V. Mnih et al., Nature, 2015

Setting: No. of states is large

Example



S: Source Node

D: Destination Node

Network — Communication N/w

Traffic N/w

Goal: Find an optimal route from S to D

Key: Dynamic optimization problem

- Weight on links is decided by no. of vehicles in buffer of each of the nodes

- Assume each node has a buffer of size 1000.

- State at time n

$$s_n = (N_1(n), \cdots, N_6(n))$$

where $N_i(n)$ = no. of vehicles at i^{th} junction at time

- Each $N_i(n)$ can take values
  - $0, 1, \cdots, 1000$

∴ Total no. of states

$$= (1001)^6 \simeq 10^{18}$$

- Storing value updates for so many states is impossible

- We need to then resort to value function approximation

* Key important quantity:

$$v_\pi(s) \approx \hat{v}(s, w)$$

where $w \in \mathbb{R}^d$ is a d-dimensional parameter.

- In real world scenarios,

$$d << |\mathcal{S}|$$

d: $\sim 50, 100$

$|\mathcal{S}|$: 10^{1000}

Examples of parameterization:

(i) Linear function approximation

$$\hat{v}(s, w) = w^T x(s)$$

$$x(s) = (x_1(s), x_2(s), \dots, x_d(s))^T$$

↓

feature of state s

(a) features can be problem dependent

(b) features based on polynomials

- Let $s = (s_1, s_2)^T$

$$x(s) = (1, s_1, s_2, s_1 s_2)^T$$

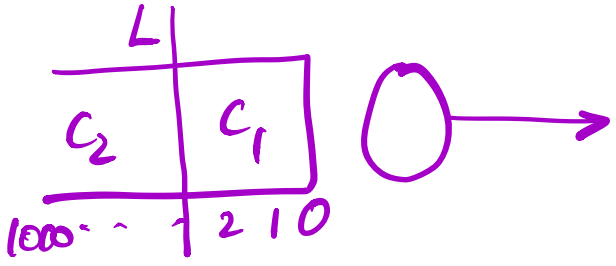
$$\hat{v}(s, w) = w^T x(s)$$

$$= w_1 + w_2 s_1 + w_3 s_2 + w_4 s_1 s_2$$

(c) features based on Fourier bases

Recall the previous example of traffic network with 6 junctions.

Consider the i^{th} junction



$L \equiv$ threshold value (arbitrarily set)

If No. of vehicles $< L$

$\Rightarrow$ Congestion is in region C_1

and if ---- $\geq L$

$\Rightarrow$ ---- C_2

Now $S_n = (N_1(n), \ldots, N_6(n))$

$N_i(n) =$ No. of vehicles at i^{th} junction.

$X(S_n) = (X_1(S_n), \ldots, X_6(S_n))^T$

where $x_i(s_n) = \begin{cases} 1 & \text{if } N_i(n) > L \\ 0 & \text{o.w.} \end{cases}$

Note: While the no. of states $\sim 10^{18}$, no. of features $= 2^6 = 64$!

Linear function approximator:

$$\hat{v}(s, w) = w_1 x_1(s) + \cdots + w_6 x_6(s)$$

(ii) Nonlinear function approximators (include neural networks)

Feed forward Neural Network as a function approximator

steps:

1. state i is encoded as

$$x = (x_1(i), \cdots, x_d(i))$$

2. X is transformed linearly through a linear layer as

$$\sum_{l=1}^{d} w(k,l)\, x_l(i), \qquad k = 1, \cdots, k$$

3. Nonlinear transformation of the above via a sigmoidal function $\sigma(\cdot)$ to obtain

$$\sigma\left(\sum_{l=1}^{d} w(k,l)\, x_l(i)\right), \quad k = 1, \ldots, k$$

$\sigma(\cdot) \rightarrow$ differentiable, monotonically increasing s.t.

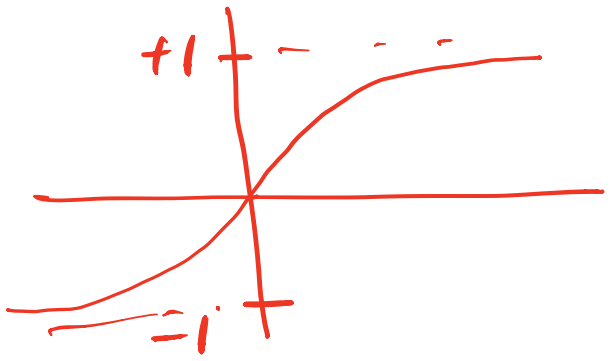
$$-\infty < \lim_{\xi \to -\infty} \sigma(\xi) < \lim_{\xi \to \infty} \sigma(\xi) < \infty$$

Examples: (i) $\sigma(\xi) = \tanh \xi$

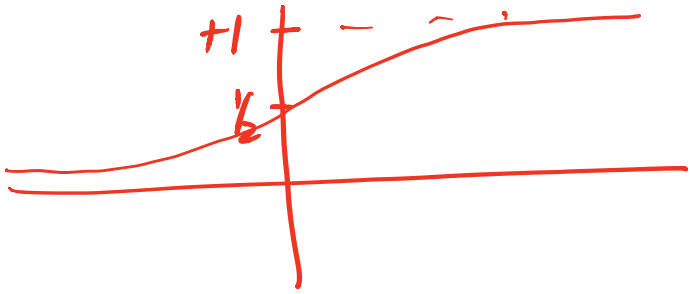
$$= \frac{e^{\xi} - e^{-\xi}}{e^{\xi} + e^{-\xi}}$$

(ii) $\sigma(z) = \dfrac{1}{1+e^{-z}}$

(i)

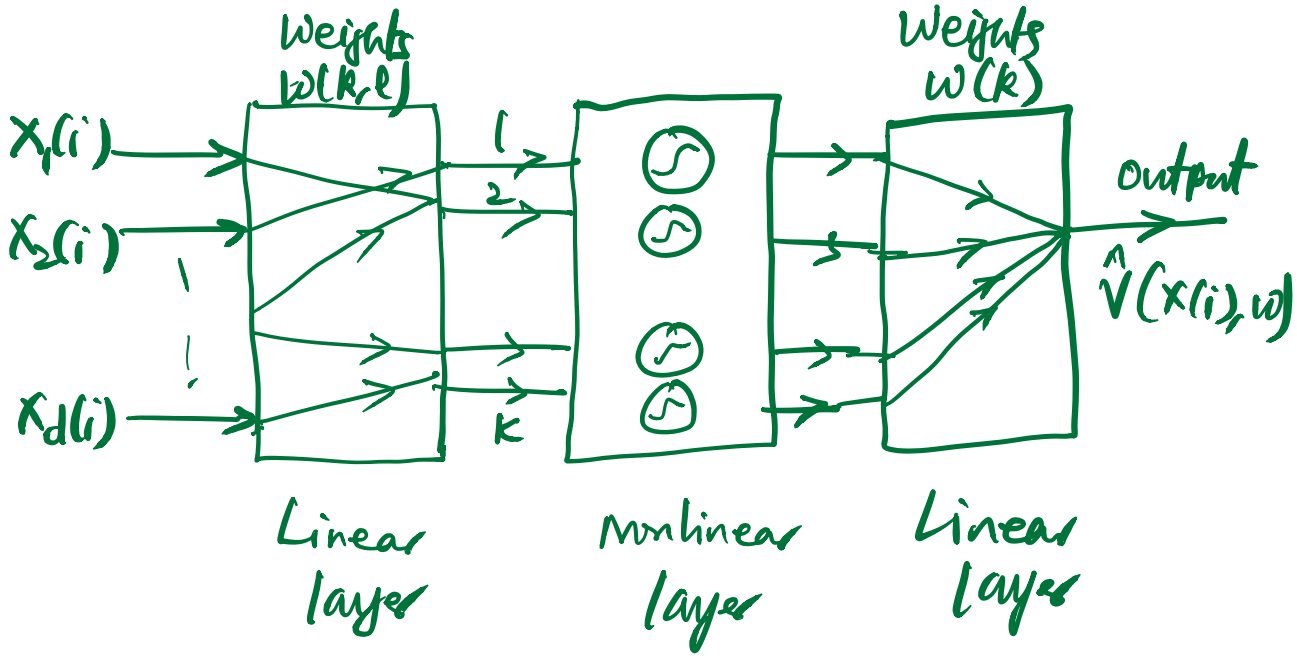


(ii)



4. Aggregate the above via another linear layer to obtain

$$\hat{V}(x(i), w) = \sum_{k=1}^{K} w(k)\, \sigma\left(\sum_{\ell=1}^{d} w(k,\ell)\, x_\ell(i)\right)$$



Weight vector

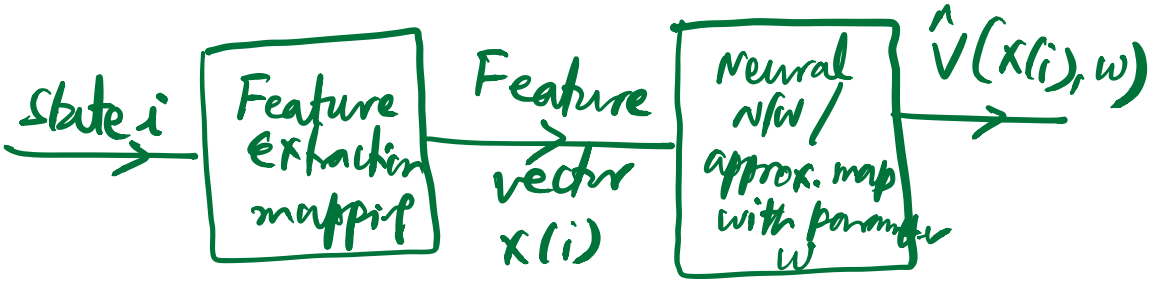
$$w = (w(k,l),\ w(k),\ k = 1, \dots, K,\ l = 1, \dots, d)$$

Normally, either $x_1(i) = 1$ or else we include 1 as an additional input

Suppose where $\sigma(0) = 0$, e.g.,

when $\sigma(z) = \tanh z$. Then,

$\hat{V}(x(i), w) = 0$ when $x(i) = 0$.



Initial problem:

Problem of prediction

- Find an approximate value function for a given policy π

Prediction Objective ($\overline{VE}$)

$$\overline{VE}(w) = \sum_{s \in S} \mu(s) \left(v_\pi(s) - \hat{v}(s, w) \right)^2$$

where $\mu(s)$, $s \in S$ is the state dist.

Goal: Find w^* that minimizes $\overline{VE}(w)$ over $w \in \mathbb{R}^d$

Update Rule:

$$w_{t+1} = w_t - \frac{1}{2} \alpha \nabla \overline{VE}(w_t)$$

step-size

$$= w_t - \frac{1}{2}\alpha \nabla \left(\sum_{s \in S} \mu(s) \left(v_\pi(s) - \hat{v}(s, w_t) \right)^2 \right)$$

$$= w_t + \alpha \sum_{s \in S} \mu(s) \left(v_\pi(s) - \hat{v}(s, w_t) \right) * \nabla \hat{v}(s, w_t)$$

$\longrightarrow (1)$

Problem 1: we don't know $\mu(s)$ $\forall s$

Since we don't know $\mu(s)$, we adopt a stochastic gradient method

$$w_{t+1} = w_t + \alpha \left(v_\pi(s_t) - \hat{v}(s_t, w_t) \right) * \nabla \hat{v}(s_t, w_t)$$

$\longrightarrow (2)$

where s_t = state observed at time t.

In steady state,

$$E[V_\pi(s_t)] = \sum_s \mu(s) V_\pi(s)$$

- As a consequence of the theory of stochastic approximation, (2) would still converge to w^*, the pt. at which $\overline{VE}(w)$ is a minimum,

  i.e., $\nabla \overline{VE}(w) = 0$.

Problem 2:

(2) involves the true value function $V_\pi(s)$

Two ways of estimating $v_\pi(s_t)$

1. Gradient Monte Carlo update

$$w_{t+1} = w_t + \alpha \left(G_t - \hat{v}(s_t, w_t) \right) \nabla \hat{v}(s_t, w_t)$$

$\longrightarrow (3)$

Here G_t = return from time t

$$G_t = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-t-1} R_T$$

where T = termination instant

2. Semi-Gradient TD algorithm:

- Recall from previous lecture, that in TD update, we use

$R_{t+1} + \gamma v_\pi(S_{t+1})$ as the target in place of G_t.

- Approximate, $v_\pi(S_{t+1}) \approx \hat{v}(S_{t+1}, w)$

Thus, we use

$R_{t+1} + \gamma \hat{v}(S_{t+1})$ in place of $v_\pi(S_t)$

Update:

$$w_{t+1} = w_t + \alpha \left(R_{t+1} + \gamma \hat{v}(S_{t+1}, w_t) - \hat{v}(S_t, w_t) \right) \nabla \hat{v}(S_t, w_t)$$

$\longrightarrow (4)$

Example:

Linear function approximator

$$\hat{v}(s, w) = w^T x(s)$$

Let $X \equiv N \times d$ matrix

$$X = \begin{bmatrix} -x(1)^T- \\ -x(2)^T- \\ \vdots \\ -x(N)^T- \end{bmatrix}$$ where

N = total no. of states, i.e.,

$N = |S|$ and d = dimension of feature vector.

Now since $X(i) = (x_1(i), \ldots, x_d(i))^T$, $i \in S$,

$$X = \begin{bmatrix} x_1(1), & x_2(1), & \cdots, & x_d(1) \\ x_1(2), & x_2(2), & \cdots, & x_d(2) \\ \vdots & \vdots & & \vdots \\ x_1(N), & x_2(N), & \cdots, & x_d(N) \end{bmatrix}_{N \times d}$$

X is called feature matrix associated with the linear function approximator.

Let $X^1 = \begin{bmatrix} x_1(1) \\ x_1(2) \\ \vdots \\ x_1(N) \end{bmatrix}$, $X^2 = \begin{bmatrix} x_2(1) \\ x_2(2) \\ \vdots \\ x_2(N) \end{bmatrix}$, $\ldots$

$$x^d = \begin{bmatrix} x_d(1) \\ x_d(2) \\ \vdots \\ x_d(N) \end{bmatrix}.$$

These vectors $x^1, x^2, \ldots, x^d$ are called basis functions of the approximator since

$$Xw = x^1 w_1 + x^2 w_2 + \cdots + x^d w_d$$

$Xw : S \to \mathbb{R}$ with $(Xw)(s) = (Xw)_s$

represents the estimate of value function in state s.

- We assume that $X^1, X^2, \cdots, X^d$ are linearly independent,

$\Rightarrow$ For $a_1, a_2, \cdots, a_d \in \mathbb{R}$, if

$$a_1 X^1 + a_2 X^2 + \cdots + a_d X^d = 0$$

$$\Rightarrow a_1 = a_2 = \cdots = a_d = 0.$$

- Note that since

$$\hat{v}(s, w) = w^T X(s),$$

we have $\nabla_w \hat{v}(s, w) = X(s)$

$$\left[\nabla_w \hat{v}(s, w) = \left(\frac{\partial}{\partial w_1} \hat{v}(s, w), \cdots, \frac{\partial}{\partial w_d} \hat{v}(s, w) \right)^T \right.$$

$$= \left(\frac{\partial}{\partial w_1} \left(w_1 x_1(s) + \cdots + w_d x_d(s) \right), \cdots, \right.$$

$$\left. \frac{\partial}{\partial w_d} \left(w_1 x_1(s) + \cdots + w_d x_d(s) \right) \right)^T$$

$$= \left(x_1(s), \cdots, x_d(s) \right)^T$$

$$= x(s) \Big]$$

Recall the semi-gradient TD update:

$$w_{t+1} = w_t + \alpha \left(R_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t) \right) \nabla \hat{v}(s_t, w_t)$$

Under linear function approximation, we have

$$w_{t+1} = w_t + \alpha\Big(R_{t+1} + \gamma w_t^T x(s_{t+1}) - w_t^T x(s_t)\Big) x(s_t)$$

$$= w_t + \alpha\Big(R_{t+1} x(s_t) + \gamma x(s_{t+1})^T w_t x(s_t) - x(s_t)^T w_t x(s_t)\Big)$$

$$= w_t + \alpha\Big(R_{t+1} x(s_t) + \gamma x(s_t)\, x(s_{t+1})^T w_t - x(s_t)\, x(s_t)^T w_t\Big)$$

$$= w_t + \alpha\Big(R_{t+1} x(s_t)$$

$$- x(s_t)\left(x(s_t) - \gamma x(s_{t+1})\right)^T w_t\Big)$$

One can show that the above converges to a pt. w^* s.t.

$$E\Big[R_{t+1} x(s_t) - x(s_t)\big(x(s_t) - \gamma x(s_{t+1})\big)^T w^*\Big] = 0$$

Let $b \triangleq E[R_{t+1} x(s_t)]$

$$A \triangleq E\Big[x(s_t)\left(x(s_t) - \gamma x(s_{t+1})\right)^T\Big]$$

Thus, w^* is the unique solution to the equation

$$b - Aw^* = 0$$

$$\text{or } w^* = A^{-1}b.$$

Paper: Human-level control th rgh Deep RL

– V. Mnih et al., Nature, 2015

Deep Q-Network algorithm

– approximate optimal action value function using deep CNN.

Fact: Q-learning with nonlinear

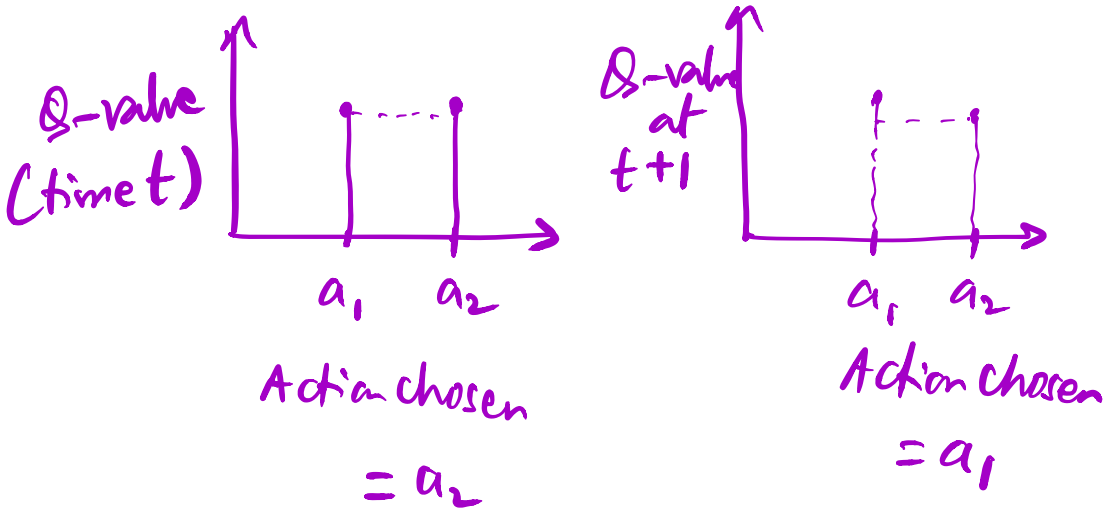
function approximators is known to diverge or be unstable.

Reasons for instability:

* Correlation between one set of observations $(s_t, a_t, r_{t+1}, s_{t+1})$ and the next $(s_{t+1}, a_{t+1}, r_{t+2}, s_{t+2})$

* Small changes to Q may result in large change in policy

Example



* Correlation between action values Q and target values $r + \gamma \max_{a'} Q(s', a')$

Innovations proposed in algorithm to tackle instability:

(i) Use Experience replay.

- Let agent's experience at time t: $e_t = (s_t, a_t, r_{t+1}, s_{t+1})$

- store e_t at any time t in a data set that grows with time.

- Let $D_t = \{e_1, e_2, \cdots, e_t\}$

- Perform Q-learning updates on sample of experience

$e = (s, a, r', s') \sim U(D)$

D: replay memory buffer

(ii) Loss function to minimize

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s') \sim U[D]} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a, \theta_i) \right)^2 \right].$$

Here $Q(s, a, \theta_i)$ ≡ parameterized Q-value function using the deep CNN with θ_i as parameter at iteration i.

- θ_i^- are Network parameters used to compute target at iteration i.

[θ_i^- is held fixed over C iterations

for stability]

Evaluation of DQN agent:

- On Atari 2600 platform (Comprises of several video games). Tested here on 49 games

- Same N/w architecture & hyperparameter values were used for games

- DQN outperforms best RL methods on 43 games

- DQN Performs comparable to a

Professional human player on all 49 games (with ≥ 75% of human score on 29 games).

- Highest score on Video Pinball game of 2539% of human score

Algorithm:

- Agent interacts with Atari emulator
- Let feasible actions at time t be

$$A = \{1, 2, \cdots, K\}$$

- Action modifies emulator's internal state & game score

- Agent observes only the current screen x_t, hence task is partially observed
- Input to algorithm at time t:

$$x_1, a_1, x_2, a_2, \dots, x_t, a_t$$

- Tasks are episodic $\Rightarrow$ All sequences in emulator terminate in a finite no. of steps

- Let $s_t = (x_1, a_1, \dots, x_{t-1}, a_{t-1}, x_t)$

$\Rightarrow$ $s_{t+1} = (s_t, a_t, x_{t+1})$

- Set $\gamma = 0.99$

Return: $G_t = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'+1}$

Let $Q_*(s,a) = \max_{\pi} E_{\pi}\left[G_t \mid s_t = s, a_t = a \right]$

Q- Bellman Equation:

$$Q_*(s,a) = E_{s'}\left[r + \gamma \max_{a'} Q_*(s',a') \mid s,a \right]$$

Training of Q- network

- Replace optimal target values

$r + \gamma \max_{a'} Q_*(s', a')$ with

approximate target values

$$y = r + \gamma \max_{a'} Q(s', a'; \theta_i^-)$$

- $L_i(\theta_i) = E_{s,a,r,s'}\left[\left(y - Q(s, a, \theta_i)\right)^2\right]$

$$\nabla_{\theta_i} L_i(\theta_i) =$$

$$-2 E_{s,a,r,s'}\left[\left(r + \gamma \max_{a'} Q(s', a', \theta_i^-)\right.\right.$$

$$\left.\left. - Q(s, a, \theta_i)\right) *\right.$$

$\nabla_{\theta_i} Q(s,a,\theta_i)\Big]$

DQN

Algorithm: use a stochastic gradient

Counterpart

$$\theta_{i+1} = \theta_i + \alpha\Big[\Big(r + \gamma \max_{a'} Q(s',a',\theta_i^-) - Q(s,a,\theta_i)\Big) * \nabla_{\theta_i} Q(s,a,\theta_i)\Big]$$

where $\alpha > 0$ is a positive step size

Implementation Aspects:

- Algorithm only stores last N experience tuples in the replay memory

- Use separate network for generating targets y_j in Q-learning update.

[y_j depends on θ_i^- & not θ_i]

- Every C updates, clone the DQN N/W to obtain target N/W $\hat{Q}$ & use $\hat{Q}$ to generate targets y_j for next C updates

- Clip the error term

$$\left(r + \gamma \max_{a'} Q(s', a', \theta_i^-) - Q(s, a, \theta_i)\right)$$

to be between -1 & $+1$

Algorithm: Deep Q-learning with Experience Replay

- Initialize replay memory D to capacity N
- Initialize action-value function Q with random weights θ
- Initialize target action-value function $\hat{Q}$

with initial weights $\theta^- = \theta$

For episode = 1, ⋯, M, do

- Initialize sequence $s_1 = \{x_1\}$ &

  preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, \cdots, T$, do

- with prob. ϵ, select a random a_t
- Else select $a_t = \underset{a}{\arg\max}\, Q(\phi(s_t), a, \theta)$

Execute action a_t in emulator, observe reward r_t & image x_t

Let $s_{t+1} = (s_t, a_t, x_{t+1})$ & preprocess

$$\phi_{t+1} = \phi(s_{t+1})$$

- Store transition $(\phi_t, a_t, r_{t+1}, \phi_{t+1})$ in D

- Sample random minibatch of transitions $(\phi_j, a_j, r_{j+1}, \phi_{j+1})$ from D

- Set $y_j = \begin{cases} r_{j+1} & \text{if episode terminates at } j+1 \\ r_{j+1} + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a', \theta^-) & \text{o.w.} \end{cases}$

- Perform a gradient descent step on

$\left(y_j - Q(\phi_j, a_j, \theta)\right)^2$ w.r.t.

the network parameters θ

- Every C steps, reset $\hat{Q} = Q$

End for

End for

A follow up paper:

Deep Recurrent Q-learning for Partially Observed Markov Decision Processes

— Matthew Hausknecht and Peter Stone, AAAI, 2015

- Use Deep Recurrent Q-Network

- Useful when the screen is flickering

[Screen is either fully revealed or fully obscured with certain probability]